

Individual Project Write-up

UTF-8 validation with processor-independent SIMD

[Student name] | Undergraduate submission | August 9, 2026

Evidence status Replace bracketed identity fields and insert final commit hashes and measured performance observations before submission.

1. Role in the Overall Project

My role was to own correctness and the second compiler-generated implementation, while my groupmate owned performance benchmarking and plots. I translated the professor's final-block question into a reproducible acceptance criterion, defined the boundary-test matrix, implemented the corrective EOF behavior, derived a validation-only Pablo kernel from u8u16, and integrated both implementations behind one command-line interface. I also led the design and implementation sections of the group report and the technical portion of the poster.

2. Leadership

- Converted an open-ended correctness concern into exact tests at B, 2B, and 3B boundaries for multiple SIMD widths.
- Established the rule that correctness must be verified before any benchmark result is accepted.
- Separated ownership cleanly: I supplied tested binaries, semantics, and interpretation criteria; my groupmate supplied reproducible timing data and plots.
- Created a benchmark handoff specifying environment capture, warmups, trials, raw CSV columns, variability, and thread scaling.
- Integrated code, report structure, poster narrative, and contribution records so the final work answers one research question.

3. Technical Contribution

3.1 Exact-block EOF correctness

The initial validator carried the prior SIMD block and correctly detected many malformed byte patterns, but it relied on a following padded block to expose required continuation bytes after a final leader. I reproduced the case where the input length is an exact multiple of the block width and the file ends with an unfinished two-, three-, or four-byte sequence. Those cases were incorrectly reported as valid.

I refactored the direct validator into a block-oriented kernel and added a narrow EOF guard expressed over the original byte stream. The guard advances legal leaders to their required continuation positions and applies Pablo's AtEOF operation to the outstanding scope. Its Add1-position result is counted with the direct kernel's errors. This makes EOF validity explicit and independent of source padding behavior.

3.2 Validation-only u8u16/Pablo implementation

I created a second implementation from the logical validation portion of u8u16. It retains leader classification, continuation scopes, scope/suffix mismatch, illegal leader checks, narrow second-byte ranges, and EOF detection. It removes all sixteen UTF-16 output streams, selectors, deletion/compression stages, P2S conversion, and file output. The application selects it with `--algorithm=pablo`; `--algorithm=direct` remains the default.

3.3 Tests and verified result

The generated test suite uses Python's strict UTF-8 decoder as an independent oracle. It checks valid and invalid inputs, sequences crossing boundaries, exact-boundary truncation, and lengths within three bytes of one and two block boundaries. Both Parabix implementations passed all 496 cases across 64-, 128-, 256-, and 512-bit block sizes.

Deliverable	Evidence to insert
Direct/EOF implementation	[final commit hash]
Pablo implementation	[final commit hash]
Differential test suite	496/496 cases passed
Report sections	Design, Implementation Details, correctness evaluation, conclusion draft
Poster sections	UTF-8 rules, designs, EOF case, technical conclusion

4. Insight and Lessons Learned

First, SIMD streaming correctness depends on termination semantics, not only on correct vector operations. A test corpus with short files can exercise padding and still miss exact-boundary behavior. Boundary lengths must therefore be treated as semantic test dimensions.

Second, processor-independent SIMD improves portability of expression but does not guarantee optimal generated code. The direct implementation avoids S2P conversion but has lower-level state and an EOF guard; the Pablo implementation expresses scopes clearly but pays for transposition. A fair evaluation must measure those representation costs rather than treating each executable as a black box.

Third, performance evidence is only meaningful when JIT/setup, file I/O, validation, and synchronization are distinguished. The most useful unexpected result from the final benchmark should be inserted here: [describe one result that contradicted the team's expectation, cite the measured data, and explain the likely compiler or memory-system cause].

5. Final Contribution Statement

My individual contribution was the correctness resolution, the additional validation-only compiler implementation, the differential boundary suite, and the technical synthesis connecting those results to the compiler-study objective. I did not produce or claim my groupmate's benchmark measurements; I defined their acceptance criteria and reviewed their interpretation for consistency with the validated implementations.